



National University of Computer and Emerging Sciences

Karachi Campus

EL-2003 Computer Organization and Assembly Language (COAL) Lab

Fall 2026

PROJECT PROPOSAL REPORT

*Depth Charge: A Real-Time Submarine Defense Game with Sonar Detection
in x86 Assembly Language using MASM and Irvine32*

Project Title	Depth Charge: A Real-Time Submarine Defense Game with Sonar Detection in x86 Assembly using Irvine32
Group ID	
Group Leader (Name & Roll No.)	Muhammad Anas – 25K-0899
Member 2 (Name & Roll No.)	Syed Alyan Hussain – 25K-0648
Member 3 (Name & Roll No.)	Ali Raza – 25K-0827
Lab Section	BCS-3C
Lab Instructor	Muhammad Owais
Submission Date	15 October 2026

Part B: Project Proposal

1. Project Title

Depth Charge: A Real-Time Submarine Defense Game with Sonar Detection in x86 Assembly using Irvine32

2. Group Information

S. No.	Name	Roll Number	Role
1	Muhammad Anas	25K-0899	Group Leader
2	Syed Alyan Hussain	25K-0648	Member
3	Ali Raza	25K-0827	Member

3. Abstract

Depth Charge is a real-time arcade game written entirely in 32-bit x86 Assembly Language (MASM) using the Irvine32 library. The player commands a submarine that moves along the bottom of an 80×25 console screen and fires torpedoes upward to destroy depth charges falling from the surface. Some of these mines are hidden: they exist in memory and can still destroy the submarine, but they are drawn only when the player activates sonar, which freezes the action for two seconds, plays a rising frequency sweep and reveals every hidden mine. The game has non-blocking keyboard input, millisecond-based frame timing, pseudo-random spawning, collision detection, explosions, sound effects, score, lives, increasing difficulty and a saved high score. The project shows how a small real-time game engine can be built from registers, arrays, structures, procedures and low-level console I/O.

4. Introduction and Problem Statement

Most lab programs so far run sequentially: they print a prompt, block on ReadInt, compute and exit. A real-time game cannot work this way. It must move many objects at a constant speed, react to a key the moment it is pressed and never stop to wait for the user. Depth Charge addresses this problem by implementing a complete game loop (poll input $\rightarrow$ update $\rightarrow$ detect collisions $\rightarrow$ render $\rightarrow$ wait for the next frame) in assembly language. The idea fits assembly well: every game object is a few bytes in memory, so each frame means walking arrays with indexed addressing, comparing coordinates with CMP and conditional jumps, and turning a (row, column) position into a memory offset with multiplication and shifts. The signature **sonar** mechanic separates the *logical state* of the game (mines that exist in memory) from its *visual state* (what is drawn), which is a core idea in computer organization.

Environment note: the game is a 32-bit Windows console program built with Irvine32, as taught in the lab. Real-mode techniques (B800h video memory, INT 16h/1Ah, speaker ports 42h/43h/61h) are not allowed in Windows user mode, so we use equivalents that are allowed: an 80×25 character/attribute buffer laid out like text-mode video memory (offset = $(\text{row} \times 80 + \text{col}) \times 2$), ReadKey for non-blocking input, GetMseconds for timing and the Win32 Beep function for sound.

5. Objectives

1. Implement a keyboard-controlled submarine (A/D or arrow keys) with boundary checking that can have up to 8 torpedoes on screen at once, using non-blocking input.
2. Spawn up to 15 depth charges at pseudo-random columns, falling faster at each level, with torpedo–charge and charge–submarine collision detection, score (+10 / +25) and 3 lives.
3. Implement sonar: hidden mines are revealed for exactly 2000 ms when SPACE is pressed, then a 10-second cooldown begins and is shown in the status bar.
4. Keep a steady 12 frames per second (80 ms per frame via GetMseconds) on any CPU, with flicker-free rendering that redraws only changed cells.
5. Provide a menu, instructions, pause, sound effects, a game-over screen and a high score saved to a file.

6. Scope of the Project

The project WILL include	The project will NOT include (out of scope)
• Single-player game in an 80 × 25 colored text console • Submarine, torpedoes, charges, hidden mines, explosions • Sonar mode with timer and cooldown; sound via Beep • Score, 3 lives, levels, status bar (HUD) • Menu, instructions, pause, game over; high-score file	• Graphics mode, bitmap sprites or mouse input • Multiplayer or network play; music/WAV audio • BIOS/DOS interrupts or direct I/O-port access • Saving and resuming a game in progress • Hand-designed levels (difficulty changes speed only)

7. Proposed Features / Modules

No.	Feature / Module	Description
1	Main Menu & Instructions	Title screen with Start / Instructions / Exit; lists controls and scoring.
2	Input Handler	Polls ReadKey each frame without blocking; maps A/D/←/→, W, SPACE, P, ESC.
3	Submarine	Moves the 'U' left/right on row 23, limited to columns 1–78.
4	Torpedoes	Fires '^' into a free slot of an 8-element array; moves up one row per frame.
5	Depth-Charge Spawner	Creates '*' at a random column (LCG + DIV); about 20% are flagged Hidden.
6	Charge Movement	Charges fall one row every N frames; N shrinks as the level rises.
7	Collision Detection	Nested loops compare X/Y (and the adjacent row) of torpedoes, charges and submarine.
8	Explosions	A destroyed charge becomes an explosion animated for 5 frames.
9	Sonar System	SPACE freezes updates, plays a 400→800 Hz sweep and shows hidden mines as '!' for 2 s.
10	Sound Effects	Different Beep tones for fire, explosion, sonar, life lost and game over.
11	Renderer & HUD	Draws the 80 × 25 buffer, redrawing only cells that changed; status bar on row 0.
12	Score, Lives & Levels	+10 per charge, +25 per hidden mine, 3 lives, level up every 200 points.
13	Game Over & High Score	Shows the final score; reads and updates highscore.dat.

8. Assembly Language Concepts to be Used

Concept	Where it is used
Registers & data movement	MOV, MOVZX, LEA: coordinates and counters in registers; ESI/EDI as array pointers.
Arithmetic	ADD/SUB/INC/DEC for movement and score; MUL for row $\times$ 80 and the LCG; DIV for modulo.
Logical & shifts	SHL for $\times 2$ cell offset and color attribute ($bg \ll 4 \mid fg$); XOR/SHR for random bits; TEST for flags.
Jumps & loops	CMP + JE/JNE/JL/JG/JA/JB for input, boundaries, collisions and states; LOOP over arrays.
Arrays & addressing modes	Direct, indirect and base-index-scale addressing, e.g. <code>[esi + ecx*TYPE GameObject]</code> .
Structures	<code>GameObject STRUCT (x, y, active, hidden)</code> for torpedoes, charges and explosions.
Procedures & stack	One PROC per module; PROTO/INVOKE with parameters, USES, PUSH/POP, LOCAL.
String primitives	REP STOSW clears the buffer, REP MOVSD saves the previous frame, REPE CMPSW finds changes.
Macros & constants	EQU for states, keys and colors; macros such as <code>mPlaySound freq, dur</code> .
Irvine32 & Win32	Gotoxy, SetTextColor, WriteChar, WriteDec, ReadKey, GetMseconds, Randomize; Beep, SetConsoleCursorInfo.
File handling	CreateOutputFile, OpenInputFile, ReadFromFile, WriteToFile, CloseFile for the high score.

9. Tools and Environment

- **Assembler & IDE:** MASM (ml.exe, 32-bit, `.model flat, stdcall`) in Visual Studio 2022, using its Registers, Memory and Watch windows for debugging.
- **Libraries & platform:** Irvine32.inc/.lib, Kernel32.lib, User32.lib (SmallWin.inc for Win32 prototypes) on a Windows 10/11 console of at least 80×25 .
- **Other:** Git/GitHub for sharing code. Any AI-assisted code will be disclosed in the final report, and all three members will be able to explain every line in the viva.

10. Proposed System Design

The program is a finite-state machine (MENU, PLAYING, SONAR, PAUSED, GAME_OVER) driven by one game loop (Figure 1). Each pass is one frame: the keyboard is polled without blocking, the current state decides which updates run, collisions are checked on the object arrays (not on the screen), the frame is drawn and `FrameDelay` waits until 80 ms have passed. Because `Beep` blocks, in-game sounds last 30 ms or less; the longer sonar sweep plays only while updates are frozen.

Every object is a 4-byte `GameObject STRUCT (x, y, active, hidden)`, stored in fixed arrays (Torpedoes $\times 8$, Charges $\times 15$, Explosions $\times 5$), and the screen is a `WORD` buffer of 80×25 cells (low byte = ASCII character, high byte = color). Hidden mines sit in the same Charges array as visible ones; only the renderer checks the hidden flag, so a hidden mine can still hit the submarine or a torpedo. Figure 2 shows the planned screen layout.

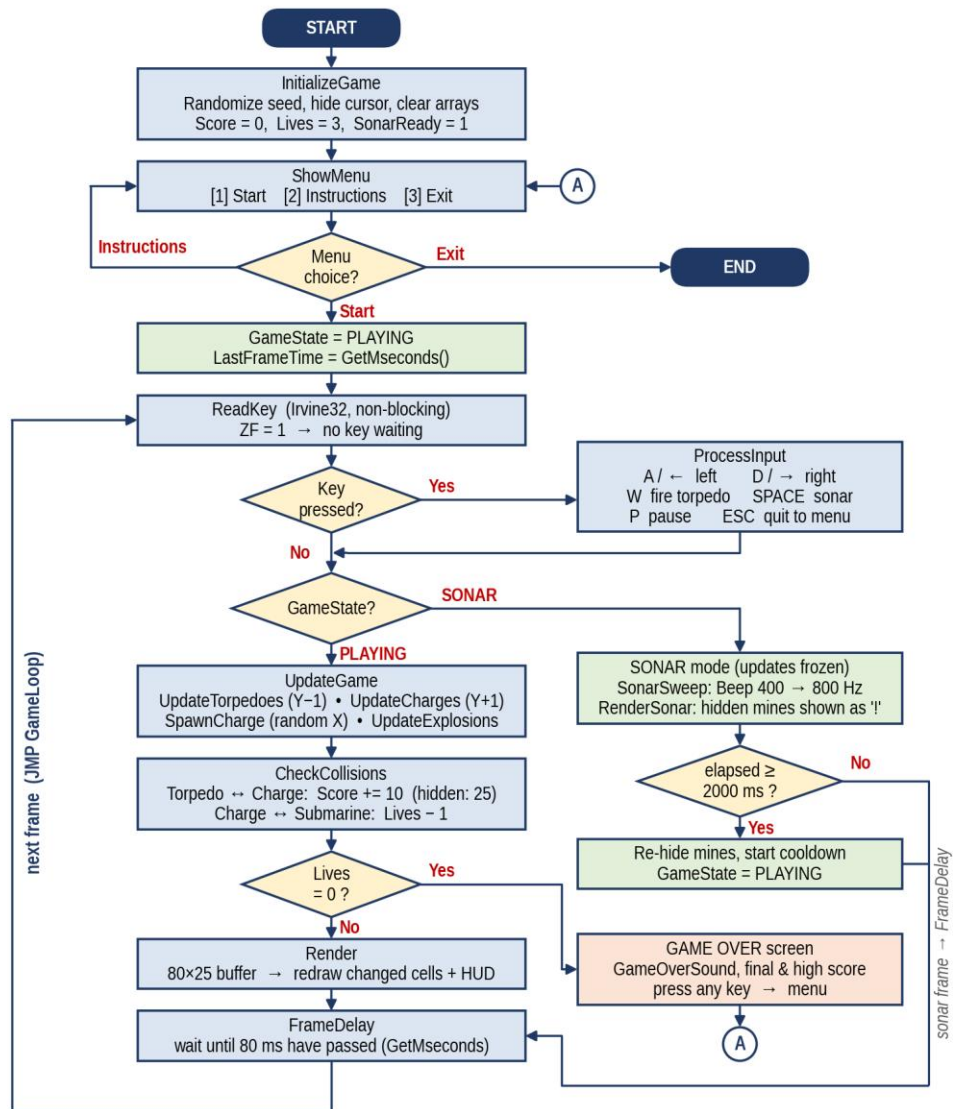


Figure 1: Main program flow and game-loop state machine

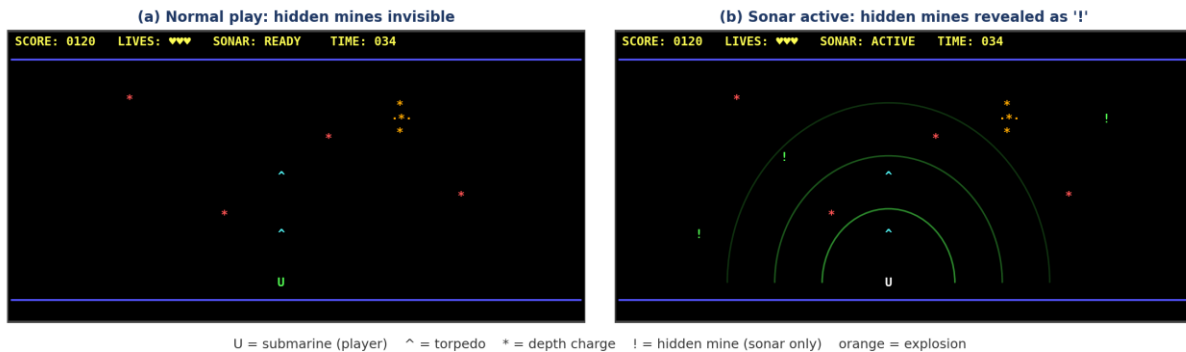


Figure 2: Main screen sketch: (a) normal play, (b) sonar mode revealing hidden mines

11. Work Division

Member	Responsibilities
Muhammad Anas (Group Leader)	Game loop and state machine, keyboard input handler, frame timing, submarine movement, torpedoes, collision detection, pause; final integration of all modules.
Syed Alyan Hussain (Member 2)	Screen buffer and renderer (changed-cell drawing), HUD/status bar, explosions, sonar system and hidden-mine rendering.
Ali Raza (Member 3)	Random number generator, depth-charge spawner and movement, score/lives/levels, high-score file, sound effects, menu/instructions/game-over screens.
All members	Testing, final report; everyone reviews the others' code to explain the whole program in the viva.

12. Tentative Timeline

Week / Dates	Task	Responsible
16 Oct – 25 Oct	Project setup, constants and structures, game loop, non-blocking input, submarine movement, 80 ms frame timing, screen buffer and renderer, menu screens, random generator.	Anas: loop, input, timing Alyan: buffer, renderer Ali: menus, random
26 Oct – 8 Nov	Torpedoes, collision detection, HUD, explosions, spawner and falling charges, score and lives.	Anas: torpedoes, collisions Alyan: HUD, explosions Ali: spawner, score, lives
9 Nov – 18 Nov	Hidden mines and sonar, pause, levels, sound effects, game-over screen, high-score file, integration.	Anas: pause, integration Alyan: sonar, hidden mines Ali: sound, levels, file
19 Nov – 22 Nov	Testing, debugging and final report	All members

13. Expected Outcome

The result will be a single executable, DepthCharge.exe, that opens a colored 80×25 console with a title menu. In play, the submarine sits at the bottom, a status bar shows score, lives, sonar status and level, and depth charges fall at random positions. The player moves with A/D or the arrow keys and fires with W; each hit shows an explosion, plays a sound and adds points. When an unseen mine destroys the submarine, the player learns to press SPACE: the game freezes, a sonar sweep sounds and hidden mines appear as green '!' marks for two seconds. After three lives are lost, a game-over screen shows the final and high score, and the player returns to the menu.

14. References

- [1] K. R. Irvine, *Assembly Language for x86 Processors*, 8th ed. Hoboken, NJ, USA: Pearson, 2019.
- [2] K. R. Irvine, "Irvine32 library and book resources." [Online]. Available: <http://asmirvine.com/>
- [3] Intel Corp., *Intel® 64 and IA-32 Architectures Software Developer's Manual, Vol. 2: Instruction Set Reference*. Santa Clara, CA, USA: Intel.
- [4] Microsoft, "Microsoft Macro Assembler reference," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/cpp/assembler/masm/microsoft-macro-assembler-reference>
- [5] Microsoft, "Beep function (utilapi.h)," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/api/utilapi/nf-utilapi-beep>
- [6] R. Nystrom, "Game loop," in *Game Programming Patterns*. Genever Benning, 2014. [Online]. Available: <https://gameprogrammingpatterns.com/game-loop.html>